

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«Национальный исследовательский университет ИТМО»
Факультет программной инженерии и компьютерной техники

«Алгоритмы и структуры данных»
отчет по блоку задач №3 (Яндекс.Контест)

Выполнил:
Студент группы Р3231
Тимошкин Р.В.
Преподаватель:
Косяков М.С.

Санкт-Петербург, 2025

Задача №10 «J. Гоблины и очереди»

Описание:

Есть очередь, в которую можно вставлять в середину, в конец, и из которой можно выходить из начала. Для каждого запроса выхода нужно найти элемент, который стоит в начале.

Доказательство:

Наивная реализация с использованием `std::deque` неэффективна из-за операции вставки в середину, имеющей сложность $O(N)$. Чтобы достичь амортизированной сложности $O(1)$ для всех операций, используем структуру данных из двух `deque`, представляющих первую и вторую половины очереди. Обычные добавления (+) идут в конец второй половины, привилегированные (*) — в конец первой или начало второй в зависимости от четности общего размера, а удаления (-) — из начала первой. После каждой операции выполняется балансировка (перемещение элемента из начала второй в конец первой половины, если размеры не сбалансированы), которая также имеет амортизированную сложность $O(1)$, поддерживая структуру и обеспечивая быструю вставку в середину.

Алгоритмическая сложность:

1. **По времени:** Сложность каждого отдельного запроса: «+ i», «* i» и «-» — amortized $O(1)$. Значит, для N запросов в лучшем и среднем (так как в худшем на каждое изменение `deque` выделяется память) случаях будет временная сложность — $O(N)$.
2. **По памяти:** Программа в общем случае хранит в очереди до N гоблинов (каждый гoblin занимает постоянное место) и некоторое постоянное число переменных (`size`, `n`, `new_goblin`, `i`, `command`, `value`), что в худшем случае требует $O(N)$ памяти.

Итого:

- По времени: $O(N)$ (линейная сложность).
- По памяти: $O(N)$ (линейная сложность).

Код:

```
1 #include <deque>
2 #include <iostream>
3 #include <vector>
4
5 namespace {
6 auto GoblinsAndQueues(int n, const std::vector<std::pair<char, int>>&
    ops) {
7     auto size = std::size_t{0};
8     auto first = std::deque<int>{};
9     auto last = std::deque<int>{};
10    auto result = std::vector<int>{};
11    result.reserve(n);
12
13    auto move_between = [&]() {
14        if (!last.empty()) {
```

```

15         first.push_back(last.front());
16         last.pop_front();
17     }
18 };
19
20 auto equalize = [&]() {
21     if (size % 2 == 0) {
22         move_between();
23     }
24 };
25
26 for (auto i = 0; i < n; ++i) {
27     auto [op, id] = ops[i];
28     if (op == '+') {
29         last.push_back(id);
30         equalize();
31         ++size;
32     } else if (op == '*') {
33         if (size % 2 == 0) {
34             first.push_back(id);
35         } else {
36             last.push_front(id);
37         }
38         ++size;
39     } else { // op == '-'
40         auto ans = first.front();
41         first.pop_front();
42         equalize();
43         --size;
44         result.push_back(ans);
45     }
46 }
47 return result;
48 }
49 } // namespace
50
51 int main() {
52     auto req_cnt = 0;
53     std::cin >> req_cnt;
54
55     auto ops = std::vector<std::pair<char, int>>{};
56     ops.reserve(req_cnt);
57
58     for (auto i = 0; i < req_cnt; ++i) {
59         char operation{};
60         std::cin >> operation;
61         if (operation == '+' || operation == '*') {
62             auto index = 0;
63             std::cin >> index;
64             ops.emplace_back(operation, index);
65         } else {
66             ops.emplace_back(operation, 0);
67         }

```

```

68     }
69
70     auto answers = GoblinsAndQueues(req_cnt, ops);
71     for (auto answer : answers) {
72         std::cout << answer << "\n";
73     }
74     return 0;
75 }

```

Задача №12 «L. Минимум на отрезке»

Описание:

По массиву из N чисел слева направо с шагом 1 движется окно размером K . Для каждого положения окна найти минимум.

Доказательство:

Наивный подход с поиском минимума в каждом окне за $O(K)$ приводит к общей сложности $O(N \cdot K)$, что неэффективно. Для оптимизации до $O(N)$ используем двустороннюю очередь (`deque`), хранящую индексы элементов текущего окна. При добавлении нового элемента i , из конца `deque` удаляются индексы элементов, значения которых не меньше `seq[i]`, так как они больше не могут быть минимумами. Из начала `deque` удаляются индексы, вышедшие за пределы текущего окна. Благодаря этому, индекс минимального элемента текущего окна всегда находится в начале `deque`, и доступ к нему осуществляется за $O(1)$. Каждое добавление и удаление индекса происходит не более одного раза, что обеспечивает общую линейную временную сложность.

Алгоритмическая сложность:

1. **По времени:** Мы считываем N чисел. В основном цикле работы с окном происходит N итераций. Удаление старых элементов, добавление нового и вывод минимума происходят за константное время, а цикл удаления *больших* элементов *за все разы* суммарно выполнится не больше N раз (больше чем есть он удалить, очевидно, не сможет), а значит получаем всего не более $N + N$ операций. Таким образом, в среднем и худшем случаях временная сложность — $O(N)$.
2. **По памяти:** Программа использует фиксированное количество переменных (`n`, `k`, `i`), а также массив из N введенных чисел и `deque` из не более чем $K < N$ элементов, поэтому сложность по памяти составляет $O(N)$.

Итого:

- По времени: $O(N)$ (линейная сложность).
- По памяти: $O(N)$ (линейная сложность).

Код:

```

1  #include <deque>
2  #include <iostream>
3  #include <vector>
4
5  namespace {
6  auto MinOnSegment(int seq_len, int window_len, const std::vector<int>&
    seq) {
7      auto deq = std::deque<int>{};
8      auto result = std::vector<int>{};
9      result.reserve(seq_len - window_len + 1);
10
11     for (auto i = 0; i < seq_len; ++i) {
12         // Use >= to handle duplicates correctly
13         while (!deq.empty() && seq[deq.back()] >= seq[i]) {
14             deq.pop_back();
15         }
16         deq.push_back(i);
17
18         // Remove index if it's outside the current window [i -
            window_len + 1, i]
19         if (deq.front() <= i - window_len) {
20             deq.pop_front();
21         }
22
23         // Start reporting minimums once the window is full
24         if (i >= window_len - 1) {
25             result.push_back(seq[deq.front()]);
26         }
27     }
28     return result;
29 }
30 } // namespace
31
32 int main() {
33     int seq_len{};
34     int window_len{};
35     std::cin >> seq_len >> window_len;
36
37     auto seq = std::vector<int>(seq_len);
38     for (auto& seq_elem : seq) {
39         std::cin >> seq_elem;
40     }
41
42     auto answer = MinOnSegment(seq_len, window_len, seq);
43     for (auto ans : answer) {
44         std::cout << ans << " ";
45     }
46     std::cout << "\n";
47
48     return 0;
49 }

```